

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Vulnerabilidades clásicas

Francisco Javier Rodríguez Lera ▪ Camino Fernández Llamas



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Identificar** familias de vulnerabilidad por su CWE.
- **Explicar** el mecanismo del fallo y sus precondiciones peligrosas.
- **Proponer** correcciones contextuales y sus límites.
- **Diseñar** pruebas positivas y negativas para cada caso.
- **Reconocer** que muchas debilidades se combinan entre sí.

Cómo se lee cada caso

Cada vulnerabilidad se presenta con la misma plantilla:

CWE. Identificador(es) y nombre de la debilidad.

Precondiciones peligrosas. Qué supuestos hacen posible el fallo.

Código vulnerable mínimo. Fragmento didáctico.

Mecanismo del fallo. Cadena causal que produce el efecto observable.

Corrección contextual. El control que lo neutraliza en un contexto declarado.

Prueba positiva y negativa. Comportamiento deseado y entrada adversaria.

Limitaciones. Qué *no* cubre la corrección.

Código vulnerable: material didáctico. Ejecución solo en entorno aislado y autorizado. Los identificadores CWE se contrastaron con el catálogo MITRE [1]; no constituyen prueba de explotación.

Tres grupos de familias

Grupo	Familia	Identificadores CWE
(a)	Memoria y enteros	CWE-242, CWE-120, CWE-121, CWE-190
(b)	Entrada y datos	CWE-20, CWE-22, CWE-78, CWE-89, CWE-502, CWE-1236
(c)	Estado, recursos y suministro	CWE-367, CWE-59, CWE-362, CWE-732, CWE-276, CWE-377, CWE-330, CWE-400, CWE-391, CWE-252, CWE-772, CWE-404, CWE-798, CWE-321, CWE-540, CWE-1104, CWE-1395

El caso transversal (CSV–PDF–XLSX) reaparece en cada grupo.

A. Memoria y enteros

A.1 Escritura sin límite: `gets` (CWE-242/120/121)

- **Precondición**: la entrada no está acotada; el atacante envía una línea más larga que el búfer.
- **Mecanismo**: `gets` escribe tantos bytes como haya en la línea, sin comprobar el tamaño del búfer; sobrescribe memoria contigua y puede corromper el flujo de control.
- **Corrección contextual**: `fgets(nombre, sizeof nombre, stdin)` acota la *longitud*.

Limitación: `fgets` no valida el *contenido*, trunca líneas largas (el resto queda en el flujo) y no sustituye un análisis de amenazas. Si se concatenan lecturas sin respetar el tamaño restante, el desbordamiento reaparece.

A.1: vulnerable y corregido

```c
char nombre[TAM];
gets(nombre);   /* CWE-120 */
```

Listing 1: Vulnerable (gets).

```c
char nombre[TAM];
fgets(nombre, sizeof nombre, stdin);
```

Listing 2: Corregido (fgets).

Prueba negativa: una línea de cien caracteres; verificar con AddressSanitizer o Valgrind si hay escritura fuera del búfer.

A.2 Desbordamiento de entero con signo (CWE-190)

- **Precondición**: se calcula con valores derivados de la entrada (tamaños, índices) sin comprobar límites.
- **Mecanismo**: `ancho * alto` en aritmética `int`; si excede `INT_MAX`, el resultado es *indefinido*. No es correcto asumir que «da la vuelta»: el compilador puede optimizar suponiendo que no ocurre. El valor se usa luego para dimensionar un búfer, habilitando un fallo de memoria.
- **Corrección contextual**: comprobar el producto antes de calcularlo (`INT_MAX / a`), bajo la precondición de operandos no negativos.

Limitación: cubre un único producto; con signos hay que tratar los cuatro cuadrantes; varias operaciones encadenadas exigen comprobar cada paso.

A.2: producto sin comprobar y comprobación previa

```c
long bytes(long e, long t) {
    return e * t;   /* CWE-190 */
}
```

Listing 3: Vulnerable.

```c
int mul_seguro(int a, int b) {
    if (a != 0 && b > INT_MAX / a)
        return -1;      /*
            desbordamiento */
    return a * b;
}
```

Listing 4: Corregido (contextual).

Negativa: 100000 100000 → vulnerable da valor indeterminado; corregido informa del desbordamiento.

¿Por qué el desbordamiento «habilita» otros fallos?

- Un desbordamiento puede dar un valor pequeño y positivo, apto para sortear una comprobación posterior. ¿Qué fallo de memoria puede desencadenar?

B. Entrada y datos

B.1 Validación de entrada incompleta (CWE-20)

- Debilidad «madre»: muchas otras son formas concretas de no validar.
- **Precondición**: el programa acepta datos de una fuente no confiable y los usa sin comprobar tipo, rango, longitud, codificación o formato.
- **Mecanismo**: `atoi` devuelve 0 tanto para «0» como para una cadena no numérica y no señala desbordamiento; el valor incorrecto se trata como válido.
- **Corrección**: `strtol` detecta ausencia de dígitos, caracteres sobrantes y `ERANGE`, y permite comprobar el rango.

Limitación: la validación debe ser coherente con el uso posterior; validar un número y usarlo como índice sin comprobar el límite superior reintroduce el fallo.

B.2 *Path traversal* (CWE-22)

- **Precondición**: el nombre de salida procede, total o parcialmente, de una fuente no confiable y se une a un directorio base sin canonicalizar ni confinar.
- **Mecanismo**: `os.path.join(dir, nombre)` no impone confinamiento; con `../../etc/cron.d/x` la escritura sale del directorio previsto.
- **Corrección**: resolver con `realpath` y comprobar que permanece dentro de la base (`commonpath`).

Limitación: confina en el momento de la comprobación, pero `realpath` no elimina por sí solo las ventanas de carrera (véase C.1); con enlaces intermedios conviene operar con descriptores de directorio.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

B.3 Inyección de comandos del SO (CWE-78)

- **Precondición**: se construye una orden para el *shell* concatenando datos no confiables; caracteres como `;`, `|`, `$()` permiten encadenar órdenes.
- **Mecanismo**: el shell reinterpreta la cadena; un nombre como `a.pdf; rm -rf /` añade una orden no prevista.
- **Corrección**: pasar la orden y sus argumentos como una *lista* sin shell (`shell=False`).

Limitación: el programa invocado puede tratar el argumento como opción (p. ej. si empieza por `-`); sigue siendo necesario validar y usar rutas confinadas.

B.4 Inyección SQL (CWE-89)

- **Precondición**: la consulta se compone concatenando datos en lugar de usar parámetros; el motor recibe la cadena final como *código*, no como *dato*.
- **Mecanismo**: un valor como `' OR '1'='1` cierra la comilla y convierte la condición en tautología; el atacante puede leer, modificar o borrar.
- **Corrección**: parametrizar la consulta; el motor la compila y trata el valor como dato.

Limitación: la parametrización protege la *inyección*; no corrige un control de acceso deficiente ni una consulta que devuelva datos indebidos. Usar parámetros en *todas* las consultas; no construir SQL dinámico con identificadores sin lista blanca.

B.5 Deserialización de datos no confiables (CWE-502)

- **Precondición**: se deserializa una fuente no confiable con un formato que permite construir objetos o ejecutar código durante la carga.
- **Mecanismo**: `pickle` puede invocar métodos durante la deserialización (`__reduce__`, `__setstate__`); un fichero manipulado ejecuta código con los privilegios del proceso, *al cargar*.
- **Corrección**: sustituir por un formato de datos sin ejecución (p. ej. `safe_load` o JSON).

Limitación: cambiar de formato reduce la superficie, pero la entrada sigue siendo no confiable; hay que limitar tamaño, validar esquema y evitar construcciones peligrosas del formato. Es un punto de fallo si se admiten «formatos adicionales».

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

B.6 Inyección de fórmulas en CSV (CWE-1236)

- **Precondición**: se exportan celdas cuyo texto empieza por =, +, − o @ sin neutralizarlas; el usuario abre el fichero con un programa que las interpreta como fórmulas.
- **Mecanismo**: el fallo viaja *con los datos* y se activa en el sistema de la víctima; `=HYPERLINK(...)` puede exfiltrar datos.
- **Corrección**: anteponer un apóstrofo (mitigación contextual).

Limitación: su interpretación depende del programa importador y de sus opciones; no es garantía universal. En XLSX/OpenPyXL el prefijo = es el caso directo; en CSV deben considerarse también TAB, CR y espacios iniciales. Aplicar en un único punto autoritativo y a *toda* celda.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

B.2: *path traversal* vulnerable y corregido

```python
import os
def ruta_salida(d, nombre):
    return os.path.join(d, nombre)  # CWE-22
```

Listing 5: Vulnerable.

```python
import os
def ruta_salida(d, nombre):
    base = os.path.realpath(d)
    cand = os.path.realpath(os.path.join(base, nombre))
    if os.path.commonpath([base, cand]) != base:
        raise ValueError("fuera del directorio")
    return cand
```

Listing 6: Corregido (contextual).

Negativa: `../../etc/passwd` y `sub/../../../x` se rechazan.

B.5: deserialización vulnerable y corregida

```python
import pickle
def cargar(f):
    return pickle.load(f)  # CWE-502
```

Listing 7: Vulnerable (pickle).

```python
import yaml
def cargar(f):
    return yaml.safe_load(f)  # sin
        ejecucion
```

Listing 8: Corregido (datos inertes).

`safe_load` sigue procesando entrada no confiable: limitar tamaño y validar esquema.

B.6: inyección de fórmulas y neutralización

```python
def celda(v):
    return v  # "=1+1" -> formula
```

Listing 9: Vulnerable.

```python
PELIGROSOS = ("=", "+", "-", "@")
def celda(v):
    if v.startswith(PELIGROSOS):
        return "'" + v   # verificar en destino
    return v
```

Listing 10: Corregido (contextual).

Contextual: depende del programa importador; aplicar a toda celda y en un único punto (DRY).

¿Dónde está el adversario en la inyección de fórmulas?

- El servidor que genera el CSV no evalúa la fórmula. ¿Quién la ejecuta y por qué es peligroso?

C. Estado, recursos y suministro

C.1 TOCTOU entre comprobar y usar (CWE-367/59)

- **Precondición**: se comprueba una propiedad de una ruta (`access`) y se usa después (`fopen`); existe una ventana en la que el atacante puede sustituir la ruta por un enlace simbólico.
- **Mecanismo**: la comprobación se hizo sobre un objeto y la operación recae sobre otro; carrera «comprobar-antes-de-usar».
- **Corrección**: una única `open` con `O_NOFOLLOW` evita la secuencia separada y rechaza enlaces en el último componente.

Limitación: `O_NOFOLLOW` protege solo el último componente; los directorios intermedios pueden seguir siendo enlaces. Fortalecer con operaciones relativas a un descriptor de directorio (`openat2` cuando exista).

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

C.1: TOCTOU vulnerable y corregido

```c
if (access(ruta, R_OK) == 0) {
    f = fopen(ruta, "r");  /* CWE-367
        */
}
```

Listing 11: Vulnerable (access + fopen).

```c
fd = open(ruta, O_RDONLY | O_NOFOLLOW
                | O_CLOEXEC);
/* O_NOFOLLOW: solo el ultimo
    componente */
```

Listing 12: Corregido (open atomico).

Negativa: componente final enlace simbólico → `ELOOP` en Linux.

C.2 Ficheros temporales inseguros (CWE-377/330)

- **Precondición**: nombre predecible en un directorio compartido, sin marca de exclusión; otro proceso puede crear antes el fichero o un enlace con ese nombre.
- **Mecanismo**: el espacio de nombres es pequeño y `random` no es criptográfico; sin `O_EXCL` la creación no es atómica respecto a la existencia previa.
- **Corrección**: `tempfile.mkstemp` genera nombre impredecible, crea de forma atómica con permisos restrictivos y devuelve un descriptor.

Limitación: no protege el contenido ni el directorio; si el directorio es escribible por terceros, el riesgo persiste aunque el nombre sea impredecible.

C.2: temporal predecible frente a `mkstemp`

```
import random
nombre = f"/tmp/x{random.randint
    (0,999)}"
open(nombre, "w")  # CWE-377
```

Listing 13: Vulnerable.

```
import tempfile
fd, ruta = tempfile.mkstemp(prefix="
    tmp_")
# nombre impredecible, creacion
    atomica
```

Listing 14: Corregido.

C.3 Concurrencia sin sincronización (CWE-362)

- **Precondición**: dos o más hilos acceden a un recurso compartido y al menos uno lo modifica; el resultado depende del entrelazado.
- **Mecanismo**: `contador++` consta de leer, sumar y escribir; si dos hilos intercalan, una actualización se pierde.
- **Corrección**: operación atómica (`atomic_fetch_add`) o mutex.

Limitación: las pruebas de concurrencia son *no deterministas*; pasar muchas ejecuciones no demuestra ausencia de carreras. Se requiere razonamiento sobre el entrelazado o herramientas específicas.

C.3: incremento no atómico y operación atómica

```c
static int contador = 0;
void *hilo(void *a) {
  for (int i=0;i<100000;i++)
    contador++;  /* CWE-362 */
  return NULL;
}
```

Listing 15: Vulnerable (leer-modificar-escribir).

```c
#include <stdatomic.h>
static atomic_int contador;
void *hilo(void *a) {
  for (int i=0;i<100000;i++)
    atomic_fetch_add(&contador,1);
  return NULL;
}
```

Listing 16: Corregido (atómico).

C.4 Permisos incorrectos (CWE-732/276)

- **Precondición**: se crea un recurso con permisos más amplios de los necesarios (0666/0777) o se confía en la `umask`.
- **Mecanismo**: el modo 0666 concede acceso a todos (limitado por la máscara); un dato sensible queda legible por terceros.
- **Corrección**: mínimo privilegio con 0600 al crear el fichero.

Limitación: el modo efectivo puede restringirse por `umask`; si el fichero ya existe, `open` no modifica sus permisos. Los permisos no protegen frente a un proceso privilegiado ni frente a un directorio padre accesible.

C.5 Errores no comprobados (CWE-391/252)

- **Precondición**: se ignora el valor devuelto por funciones que pueden fallar (apertura, lectura, asignación); la operación falla y el programa sigue con estado inválido.
- **Mecanismo**: si `fopen` devuelve `NULL`, usar `f` es comportamiento indefinido; si `fread` lee menos, se procesa un búfer incompleto.
- **Corrección**: comprobar cada retorno y actuar en consecuencia.

Limitación: comprobar no define la política de recuperación (abortar, reintentar, degradar); lo inadmisible es ignorar el fallo.

C.6 Fugas de recursos (CWE-772/404)

- **Precondición**: un recurso se adquiere y una ruta de ejecución lo abandona sin liberarlo (retorno anticipado o excepción).
- **Mecanismo**: el retorno temprano abandona el objeto sin cerrar; en un bucle, los descriptores se agotan y el programa falla.
- **Corrección**: gestor de contexto `with`, que cierra en todas las rutas.

Limitación: cubre este recurso; otros (conexiones, cerrojos) requieren su propio tratamiento. No detecta fugas *lógicas* (memoria retenida).

C.6: fuga de recurso y gestor de contexto

```python
def leer(ruta):
    f = open(ruta)
    if not f.readline():
        return None    # CWE-772: f
            sin cerrar
    return f.read()
```

Listing 17: Vulnerable.

```python
def leer(ruta):
    with open(ruta) as f:
        if not f.readline():
            return None  # se cierra
                igual
        return f.read()
```

Listing 18: Corregido.

C.7 Secretos embebidos (CWE-798/321/540)

- **Precondición**: el código contiene una credencial o clave como literal; el fuente es accesible (repositorio, binario, historial).
- **Mecanismo**: el secreto forma parte del artefacto; cualquiera que lea el código o el historial obtiene acceso. Los asistentes tienden a generar literales plausibles.
- **Corrección**: extraer el secreto del código y obtenerlo del entorno o de un gestor de secretos.

Limitación: las variables de entorno también pueden filtrarse (volcados, procesos, registros); exige rotación y mínimo privilegio. Una vez en el historial, la rotación es obligatoria.

C.7: secreto embebido y secreto en el entorno

```
API_KEY = "sk-live-3f9a-ejemplo"  # CWE-798
def llamar(u):
    return get(u, headers={"Auth": API_KEY})
```

Listing 19: Vulnerable.

```
import os
def llamar(u):
    clave = os.environ["API_KEY"]
    return get(u, headers={"Auth": clave})
```

Listing 20: Corregido.

C.8 Dependencias vulnerables o sin mantener (CWE-1104/1395)

- **Precondición**: se incorporan dependencias sin fijar versiones, sin revisar avisos y sin plan de actualización; una de ellas tiene una vulnerabilidad explotable en el uso que se hace.
- **Mecanismo**: sin versiones fijadas, una instalación puede traer una versión con un fallo conocido; sin seguimiento, permanece. La superficie crece con cada dependencia.
- **Corrección**: fijar versiones y ejecutar análisis de composición (SCA).

Limitación: fijar no basta; hay que actualizar al aparecer un aviso y fijar también las transitivas. La gestión de dependencias es un proceso continuo.

El caso transversal, familia por familia

Punto de fallo	CWE	Control contextual
Rutas de salida no confinadas	CWE-22	Comprobar ruta y abrir respecto a directorio confiable
Ventana validar-abrir	CWE-367/59	Apertura relativa a directorio; `O_NOFOLLOW` solo el componente final
Fórmulas en celdas	CWE-1236	Tratar como texto y verificar en consumidor declarado
Temporales predecibles	CWE-377/330	`mkstemp` con directorio controlado
Formatos adicionales	CWE-502	Formato sin ejecución (`safe_load`)
Dependencias del conversor	CWE-1104/1395	Fijar versiones y escanear con SCA

Son controles contextuales, no garantías universales; cada CWE debe justificarse para la debilidad observable.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

Síntesis del capítulo

- Las vulnerabilidades se agrupan en familias con CWE verificables: memoria y enteros; entrada y datos; estado, recursos y suministro.
- Cada caso sigue la misma plantilla: CWE, precondiciones, código, mecanismo, corrección contextual, pruebas y limitaciones.
- Muchas debilidades se **combinan**: un desbordamiento habilita un fallo de memoria; un TOCTOU anula una validación de ruta; una dependencia vulnerable compromete código propio correcto.
- La corrección es siempre **contextual**, nunca «universalmente segura».

Ningún conjunto finito de pruebas demuestra seguridad. La verificación combina análisis estático y dinámico, revisión y modelado de amenazas (ASVS [3], SSDF [2]).

Actividad

1. Elija tres familias y construya, para cada una, una prueba negativa que provoque el fallo y una que lo rechace.
2. Complete la tabla del caso transversal con dos puntos de fallo adicionales y su control.
3. Explique por qué una dependencia vulnerable puede comprometer código propio que es correcto.
4. Diseñe una variante que introduzca un fallo TOCTOU a partir de una comprobación de ruta previa.

¿Qué familia es más fácil de olvidar?

- ¿Por qué los fallos de *estado y recursos* (TOCTOU, temporales, permisos, errores) son más difíciles de ver en una revisión que los de inyección?

Referencias I



MITRE Corporation.

Common weakness enumeration (cwe).

Catálogo de referencia; los identificadores deben justificarse por caso.



National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.



OWASP Foundation.

Application security verification standard (asvs), 2025.

Guía de referencia (versión estable 5.0.0, 2025).